

SpeakSpace Local

Student Onboarding & Resource Document

University of Bristol MSc Computer Science Summer Project

Project period: 01 June 2026 to 01 September 2026

Prepared for student groups working on the SpeakSpace project

One-line project frame

Build a fresh desktop and mobile R&D prototype inspired by SpeakSpace, using local models for speech-to-text, language intelligence, translation, and text-to-speech experiments, without using production SpeakSpace screens, code, or active APIs.

1. Purpose of this document

This document gives the common context, constraints, resources, and tentative direction for the SpeakSpace Local student project. All three student groups are working on the same project theme, but each group should work independently and define its own technical plan, work distribution, and milestones after discussion with its supervisor and Alpha AI contacts.

- **Use this document as context, not a fixed specification.** The project is intentionally R&D-oriented, and teams are expected to make informed technical choices.
- **Do not clone the current product.** The current SpeakSpace app is a reference for intent and workflows, not a UI or codebase to copy.
- **Prioritize feasible local execution.** The work should be grounded in models, frameworks, and techniques that can realistically run on desktop and/or mobile hardware.

2. What is SpeakSpace?

SpeakSpace is a voice-first productivity platform that captures spoken ideas, conversations, meetings, lectures, and working notes, then converts them into structured and actionable outputs. The broader product vision is not simply transcription. It is speech-to-action: turning voice into notes, summaries, tasks, reminders, translations, and interactive note intelligence.

Core product capabilities to understand

- Voice recording and real-time transcription.
- Multilingual voice note support across a wide range of languages.
- Summaries, key points, and structured notes generated from voice or text.

- Task and action-item extraction from spoken input.
- Reminder and calendar-style intent recognition from natural language.
- AskAI-style interaction over notes and threads for clarification, Q&A, brainstorming, and deeper analysis.
- Secure workspaces, note organization, pinned notes, and dashboard views.
- Business workflow direction through custom pipelines, webhooks, and organization-level workflow automation.

Important distinction

SpeakSpace should be understood as a voice intelligence layer: it listens, understands, structures, and helps users act. SpeakSpace Local explores how much of that intelligence can move from cloud/API dependency to local and edge environments.

3. Why SpeakSpace, and why now?

Speech is one of the most natural forms of human input, but most digital systems still treat spoken information as either audio storage or plain transcription. The opportunity is to move from passive capture to structured productivity.

- **The problem is real.** Important information is often spoken in meetings, lectures, consultations, calls, brainstorms, and informal conversations, but is rarely captured in structured form.
- **The timing is strong.** Local model runtimes, small language models, speech models, and edge AI tooling are improving rapidly.
- **Privacy is becoming central.** Many real use cases in healthcare, legal, enterprise, research, and education cannot rely on unrestricted cloud processing.
- **Mobile and desktop workflows matter.** Voice productivity is useful only if it fits real user environments: phones, laptops, and offline contexts.
- **The product gap is clear.** Many tools transcribe. Fewer tools convert voice into tasks, summaries, translations, structured context, and follow-up workflows.

4. Current build available for reference

Mobile applications

The current SpeakSpace product is available as iOS and Android applications. Students may review the product experience to understand the current workflow direction, feature categories, and product philosophy.

- Use the apps as reference for product intent and user flow possibilities.
- Do not recreate the current app screens exactly.
- Do not assume access to the production codebase, databases, or internal APIs.
- Do not use production customer data.

Web application

The web application is primarily informative and does not currently replicate the mobile notes experience. However, it includes a custom workflow module where users or organizations can create custom pipelines and link those workflows to their account on the phone application. This is useful context for understanding the long-term product direction: voice capture can become a trigger for custom productivity workflows.

What students should extract from the current build

- Which user flows are valuable enough to bring into a local-first prototype.
- Which features can be simplified for R&D without losing the core product value.
- What local-first architecture changes would be required if cloud services were removed.
- Which parts of the user experience should be redesigned for desktop and non-published mobile prototypes.

5. What is SpeakSpace Local?

SpeakSpace Local is a research-led prototype project. The aim is to explore whether a SpeakSpace-like experience can be rebuilt as a fresh local-first desktop and mobile application using local models, local storage, and edge-oriented architecture.

The project is

- A new prototype application with its own UI/UX.
- A desktop + mobile exploration, with mobile not intended for public release.
- A local model integration experiment using STT, LLM/SLM, translation, and TTS components.
- An R&D-by-product exercise: the prototype should teach us what is feasible, what is slow, what is usable, and what needs future work.

The project is not

- A production extension of the live SpeakSpace app.
- A requirement to copy the current mobile screens.
- A cloud/API-first implementation.
- A feature-complete product launch.
- A polished enterprise product with production-grade security, billing, or infrastructure.

5.1 Connection to the Original Project Brief

The project selected by the student teams was submitted as **SpeakSpace-Local: A Local-First AI Productivity Assistant**. That original brief described a system that explores how local AI can derive meaning from user input and support everyday knowledge work such as:

- capturing notes and voice inputs,
- summarising content,

- extracting tasks and action items,
- understanding intent from free-form input,
- organising information into spaces, projects, or work contexts,
- retrieving and presenting context later,
- and using local AI models instead of relying on remote cloud APIs.

This onboarding document keeps the same core direction, but makes the implementation expectations more concrete for the summer project. Each team should attempt a fresh desktop + non-published mobile prototype, explore local models for STT, LLM/SLM-based text generation or translation, and TTS, and focus on a realistic R&D prototype rather than a production-grade product.

- A useful prototype may use an interaction style such as a **chat interface, command palette, timeline, note library, project/space view, or another interface chosen by the team**. The exact design is open for student exploration.
- The architecture should be modular enough that future models or capabilities can be added without rewriting the whole application. This includes clear handling of:
 - model loading,
 - prompt construction,
 - response handling,
 - local storage,
 - workspace/project structure,
 - and graceful fallback when models are slow, unavailable, or unsuitable for a target device.

6. Core project direction

Technical direction

Attempt a desktop application and a non-published mobile application that use local models for speech-to-text, language generation/translation, and text-to-speech or voice-response experiments.

Model components to explore

Component	Purpose	Example capability	What to measure
STT	Speech-to-text / transcription	Record or upload audio and convert it into text locally	Accuracy, latency, memory use, language support
LLM / SLM	Text intelligence	Summaries, key points, translation, rewriting, structured notes	Output quality, speed, token cost locally, model size
TTS	Speech synthesis / voice response	Convert generated text back into speech where useful	Voice quality, latency, package size, device support
Local storage	Offline-first persistence	Save notes, transcripts, summaries, and outputs locally	Reliability, privacy, portability, data model clarity
UI / UX	Usable desktop and mobile experience	Capture, process, view, retrieve, and reuse notes	Clarity, speed, device fit, error handling

7. Suggested feature directions

Teams may adopt or reinterpret features from the existing SpeakSpace app. It is recommended to explore options and possibilities rather than simply reproducing the current feature list.

Minimum useful flow

1. User records or imports audio, or enters text manually.
2. Local STT converts speech into transcript text where audio is used.
3. Local LLM/SLM generates a summary, key points, or transformed version of the note.
4. Optional translation is generated locally where feasible.
5. Optional TTS converts selected output into spoken audio.
6. The app stores the original note, transcript, and generated outputs locally.

Feature ideas for teams to consider

- Note capture and transcript view.
- Summary and key point generation.
- Translation between selected languages.
- Ask-style question answering over a single note or local note set.
- Task or action-item extraction.
- Simple local search over transcripts and summaries.
- Offline/local-first note library.
- Voice playback of generated outputs through TTS.
- Latency dashboard or model comparison panel for research visibility.

Workspace Semantics

Students should think carefully about how information is represented inside the app. At minimum, the prototype should define how the following are stored and connected:

- notes,
- transcripts,
- summaries,
- generated outputs,
- tasks or action items,
- projects / spaces / folders,
- and optional user questions or Ask-style interactions.

This does not need to be complex, but it should be clear. The system should not only generate outputs; it should also show how those outputs can be organised, retrieved, and reused later.

8. Requirements and constraints

- **Desktop app:** Each team should attempt a functional desktop application experience using local models or local model runtimes.
- **Mobile app:** Each team should attempt a non-published mobile application experience. It does not need to match desktop maturity, but the mobile path must be explored and documented.
- **Local-first:** The core prototype should not depend on active SpeakSpace APIs or cloud-only intelligence.
- **Fresh UI/UX:** Design a new interface suitable for this prototype. Current SpeakSpace screens are reference material only.
- **No production access:** Do not use the production codebase, production APIs, customer data, or internal databases.
- **Model and licence documentation:** Every third-party model, library, framework, dataset, or asset should be documented with licence considerations.
- **Feasibility over hype:** Tool choices should be justified with evidence: test results, official documentation, benchmarks, or small prototypes.
- **AI-assisted coding allowed:** Tools such as Codex, Claude Code, Cursor, or similar may be used, but generated code must be reviewed, tested, and understood by the team.

9. Tentative plan for the project period

This is a suggested structure only. Each team should propose its own plan, role split, technical milestones, and meeting rhythm after initial research and supervisor discussion.

Period	Tentative focus	Suggested outputs	Key decisions
Week 1-2	Discovery, app architecture, model/runtime shortlist, UI direction	Architecture sketch, UI wireframes, repository setup, initial risk list	Tech stack, desktop/mobile approach, baseline model choices

Week 3-4	First working app flow	Desktop/mobile shell, local text or audio input, first summary output in UI	How to connect model runtime to app cleanly
Week 5-7	Core local intelligence	Local STT, local summarization, translation or structured generation, local storage	Which models are good enough for continued work
Week 8-10	TTS, performance, and edge feasibility	TTS experiment, latency measurements, CPU/GPU comparison, mobile feasibility notes	What can run where, and what must be simplified
Week 11-13	Integration, stabilization, demo, handover	Final prototype, demo script, technical report, setup instructions, future roadmap	What the project proves and what remains open

10. Suggested workstream options for students

Teams should decide their own internal distribution within the same group. The following workstreams are a practical starting point for planning.

- **Product architecture and integration:** Defines module boundaries, data flow, repository structure, and integration plan.
- **Desktop application:** Builds the desktop app shell, local workflow screens, and desktop packaging path.
- **Mobile application:** Builds the non-published mobile prototype and explores local model constraints.
- **Audio and STT pipeline:** Handles recording, audio preprocessing, local transcription, and STT benchmarking.
- **LLM/SLM intelligence pipeline:** Handles summarization, translation, text generation, prompts, structured outputs, and evaluation.
- **TTS / voice response:** Explores speech synthesis or voice output from generated content.
- **Performance, testing, documentation:** Tracks latency, memory, device compatibility, reliability, setup instructions, and final reporting.

11. Expected outcomes

Prototype outcomes

- Functional desktop prototype demonstrating at least one local AI-powered note workflow.
- Non-published mobile prototype or mobile feasibility build showing how the same concept could work on-device.

- Local STT, LLM/SLM, and TTS exploration, even if not all components reach equal maturity.
- Fresh UI/UX that communicates the local-first note intelligence concept clearly.

Technical outcomes

- Architecture diagram and explanation.
- Documented model/runtime choices and reasons for selection.
- GPU vs CPU observations where applicable.
- Desktop vs mobile feasibility notes.
- Latency, quality, memory, package-size, and user-experience trade-offs.
- Clear explanation of what works, what does not, and what should be explored next.

Submission / handover outcomes

- Source code repository with clear structure.
- Setup and run instructions.
- Demo video or live demo flow.
- Final presentation aligned with university requirements.
- Final written handover / technical report.

12. Quality and evaluation lens

The strongest outcome is not the group with the largest feature list. The strongest outcome is the group that produces a working, understandable, and honest edge-AI prototype with clear evidence of technical trade-off thinking.

- **Functionality:** Does the app demonstrate a usable end-to-end workflow?
- **Local execution:** Does the core intelligence run locally, or is there a clear reason where it does not?
- **Cross-platform thinking:** Is there meaningful progress on both desktop and mobile?
- **Model integration:** Are STT, LLM/SLM, and TTS choices integrated thoughtfully rather than bolted on?
- **Latency awareness:** Are performance limits measured and discussed clearly?
- **UX clarity:** Can a user understand what the prototype does without developer explanation?
- **Documentation:** Can another technical team understand, run, and extend the work?

13. Technical exploration guidance

The technology stack is not prescribed. Teams should research current libraries, frameworks, model runtimes, packaging approaches, and mobile deployment techniques. The following are example areas to investigate, not mandatory selections.

Area	Candidate directions to research	Selection questions
------	----------------------------------	---------------------

Desktop app	Tauri, Electron, Flutter desktop, native desktop, Python UI wrappers	Packaging? Model runtime access? Performance? Cross-platform effort?
Mobile app	Flutter, React Native, native Android/iOS, lightweight local inference wrappers	Can the model run on-device? What needs to be simplified?
Local LLM/SLM	Ollama, llama.cpp, ONNX Runtime, GGUF/quantized models, small instruction models	Latency? Memory? Licence? API bridge to app?
STT	Whisper/whisper.cpp, native/mobile speech libraries where local, ONNX-based STT	Accuracy? Language support? Streaming? CPU/mobile viability?
TTS	Lightweight TTS engines and local speech synthesis options	Voice quality? Model size? Real-time feasibility? Platform support?
Data/storage	SQLite, local files, embedded vector search if needed	Privacy? Simplicity? Search/retrieval? Backup/export?
AI-assisted coding	Codex, Claude Code, Cursor, or similar coding assistants	Can the team explain, test, and maintain generated code?
Selection principle Choose tools because they are viable for the prototype, not because they are popular. A smaller model that runs reliably on a target device is more valuable than a larger model that only works in ideal conditions.		

14. First meeting preparation

Before or during the first planning cycle, each team should prepare answers to the following questions:

- What is the smallest useful user flow we can build first?
- What desktop and mobile frameworks are we considering, and why?
- Which local model/runtime options will we test first for STT, LLM/SLM, and TTS?
- What hardware do team members have available for testing?
- How will we measure latency, quality, and device feasibility?
- What can be completed confidently by the mid-project point?
- What will we document from week one so the final handover is not rushed?

15. Reference links and materials

Reference materials to review:

- SpeakSpace current product website: www.speakspace.co
- SpeakSpace mobile apps: Android and iOS app listings shared with the project team.
- Internal SpeakSpace product notes and roadmap materials shared for context.

- University project requirements and submission guidance from the MSc Software Engineering unit.

Technical documentation starting points

- OpenAI Codex developer page: <https://developers.openai.com/codex>
- Ollama documentation / model library: <https://ollama.com/>
- llama.cpp build documentation: <https://github.com/ggml-org/llama.cpp/blob/master/docs/build.md>
- whisper.cpp repository: <https://github.com/ggml-org/whisper.cpp>
- ONNX Runtime documentation: <https://onnxruntime.ai/docs/>
- ONNX Runtime mobile tutorials: <https://onnxruntime.ai/docs/tutorials/mobile/>

16. Final reminder

What good looks like

A good final project does not need to be production-ready. It should show a credible local-first desktop/mobile prototype, thoughtful model integration, honest performance findings, and a clear path for future R&D.